PROJEKT APLIKACJI MOBILNEJ

Premier League App

Opis aplikacji, ekrany, plan implementacji i architektura

Platforma	Android (Kotlin, Jetpack Compose)
Źródło danych	football-data.org – API v4, darmowy plan
Architektura	MVVM, podejście offline-first
Etap	0 – projekt i dokumentacja
Data	październik 2026

1. Wstęp i zarys projektu
2. Funkcje aplikacji
3. Ekrany aplikacji
4. Plan implementacji – vertical slices
5. Dane offline – podejście cache-first
6. Mocki jako zabezpieczenie projektu
7. Technologie
8. Architektura MVVM – diagramy

1. Wstęp i zarys projektu

Premier League App to natywna aplikacja na Androida dla kibiców angielskiej Premier League. Łączy dwie rzeczy: aktualne informacje o lidze (wyniki, terminarz, tabela) oraz osobisty **Dziennik kibica**, w którym użytkownik zapisuje wspomnienia z obejrzanych meczów.

Cel projektu

Celem jest zbudowanie kompletnej, nowoczesnej aplikacji mobilnej w Kotlinie, która pokazuje dobre praktyki tworzenia aplikacji na Androida: architekturę MVVM, pracę z zewnętrznym REST API, lokalną bazę danych, działanie bez dostępu do internetu oraz korzystanie z funkcji urządzenia – aparatu, mikrofonu, galerii i lokalizacji.

Zakres

- **Część ligowa** – dane o Premier League pobierane z API football-data.org: ekran startowy z podsumowaniem, pełna tabela ligowa i lista meczów z podziałem na kolejki.
- **Dziennik kibica** – dane tworzone przez użytkownika i przechowywane na telefonie: wspomnienia przypisane do meczów, ze zdjęciami, filmami, notatką głosową i lokalizacją.

Źródło danych

Dane ligowe pochodzą z API **football-data.org** (wersja v4). Darmowy plan obejmuje Premier League, tabele ligowe i terminarze, z dwoma ograniczeniami, które wpływają na projekt:

- wyniki są udostępniane z opóźnieniem (brak wyników na żywo),
- limit wynosi **10 zapytań na minutę**.

Z tego powodu aplikacja nie odpytuje API przy każdym wejściu na ekran, tylko trzyma dane w lokalnej bazie i odświeża je według reguły opisanej w rozdziale 5. Każde zapytanie wymaga klucza API w nagłówku `X-Auth-Token`. Klucz jest przechowywany poza repozytorium (`local.properties` → `BuildConfig`).

Wymagania projektu i ich realizacja

Wymaganie	Jak jest spełnione w aplikacji
Komunikacja z zewnętrznym API	Pobieranie tabeli, meczów i informacji o lidze z football-data.org (Retrofit).
Geolokacja	Przy wspomnieniu można jednorazowo pobrać pozycję z GPS i zamienić ją na adres (Geocoder).
Plik audio	Nagrywanie i odtwarzanie notatki głosowej do wspomnienia.
Filmik	Odtwarzanie filmów dodanych do wspomnienia (odtwarzacz Media3).
Wgrywanie własnych zdjęć i filmów	Dodawanie multimediów z galerii (Photo Picker) lub bezpośrednio z aparatu.

2. Funkcje aplikacji

Aplikacja ma cztery zakładki dostępne z dolnego paska nawigacji: **Start**, **Tabela**, **Mecze** i **Kibic**. Poniżej zestawienie tego, co użytkownik może zrobić, wraz z ekranem, na którym to robi.

Co może zrobić użytkownik	Ekran	Uwagi
Zobaczyć ostatnie wyniki, najbliższe mecze i czołówkę tabeli w jednym miejscu	Start	Ekran-podsumowanie, linki do pełnych widoków.
Sprawdzić, kiedy dane były ostatnio aktualizowane	Start, Tabela, Mecze	Informacja „Zaktualizowano X min temu”.
Odświeżyć dane gestem przeciągnięcia w dół lub przyciskiem	Start, Tabela, Mecze	Wymusza pobranie z API (pull-to-refresh).
Przeglądać pełną tabelę 20 drużyn	Tabela	Strefy: Liga Mistrzów, Liga Europy, spadek.
Przełączać tabelę: ogółem / u siebie / na wyjeździe	Tabela	API zwraca wszystkie trzy warianty naraz.
Przeglądać mecze wybranej kolejki, pogrupowane po dniach	Mecze	Przełączanie kolejek strzałkami.
Korzystać z aplikacji bez internetu	Wszystkie	Dane z lokalnej bazy (rozdział 5).
Przeglądać listę swoich wspomnień z meczów	Kibic – lista	Pusty stan z zachętą przy pierwszym użyciu.
Dodać wspomnienie do wybranego meczu	Kibic – nowe wspomnienie	Mecz wybierany z listy meczów ligi.
Dodać zdjęcia i filmy z galerii lub z aparatu	Kibic – nowe wspomnienie	Photo Picker nie wymaga uprawnień.
Nagrać i odsłuchać notatkę głosową	Kibic – nowe wspomnienie, szczegóły	Uprawnienie do mikrofonu przy pierwszym nagraniu.
Zapisać miejsce dodania wpisu (GPS) i poprawić adres ręcznie	Kibic – nowe wspomnienie	Lokalizacja opcjonalna, pobierana jednorazowo.
Obejrzeć wspomnienie: galerię, film, notatkę, mecz i miejsce	Kibic – szczegóły	Edycja i usuwanie z menu.

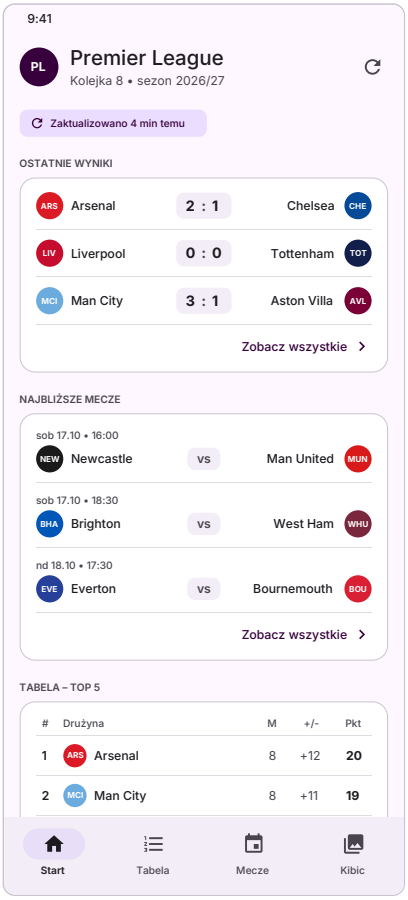
> **Dlaczego dziennik, a nie osobne funkcje?** Wymagania dotyczące geolokacji, audio, wideo i wgrywania plików zostały połączone w jedną spójną funkcję. Każde wspomnienie to pamiątka z konkretnego meczu – stąd zdjęcia i filmy z trybun, komentarz głosowy oraz miejsce, w którym kibic oglądał mecz. Dzięki temu każde wymaganie ma naturalne uzasadnienie w aplikacji, a całość łatwo pokazać jednym scenariuszem.

3. Ekrany aplikacji

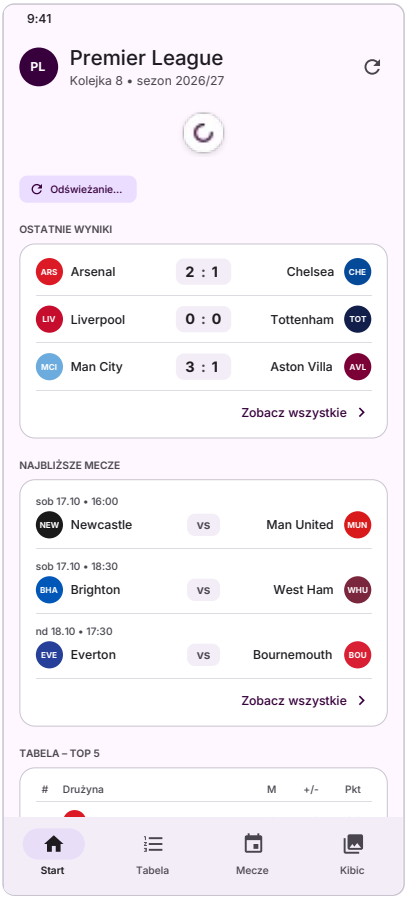
Makiety zostały przygotowane w Figmie w oparciu o Material 3 – ten sam system komponentów, który jest dostępny w Jetpack Compose. Każdy ekran ma szkielet `Scaffold` z górnym paskiem i przyklejoną do dołu nawigacją; przewija się tylko zawartość pomiędzy nimi. Dane w makietach (wyniki, punkty) są przykładowe.

3.1 Start – podsumowanie ligi

Ekran główny zbiera najważniejsze informacje: numer bieżącej kolejki, trzy ostatnie wyniki, trzy najbliższe mecze oraz pięć pierwszych drużyn w tabeli. Linki „Zobacz wszystkie" i „Pełna tabela" prowadzą do pozostałych zakładek. Chip pod nagłówkiem informuje o wieku danych, a przycisk w prawym górnym rogu (lub przeciągnięcie w dół) odświeża dane.



Start – dane z cache
standardowy widok

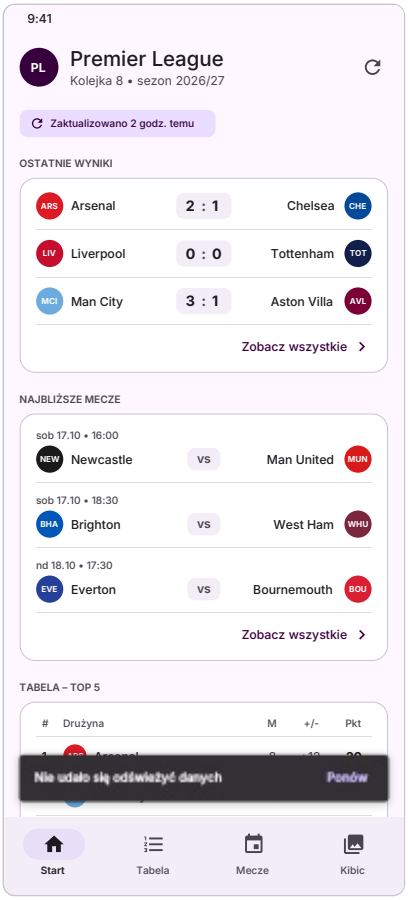


Start – odświeżanie
dane widoczne podczas pobierania

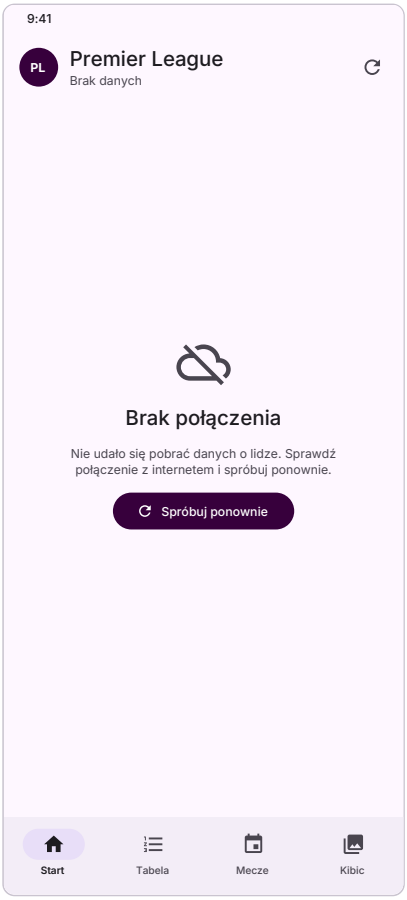


Start – pierwsze ładowanie
szkielety (skeleton), baza jest pusta

Ekran został zaprojektowany we wszystkich stanach, które może przyjąć. Pokrywają się one 1:1 ze stanami `UiState` w ViewModelu i z działaniem cache opisanym w rozdziale 5.



Start – błąd, dane z cache
stare dane + snackbar „Ponów"



Start – błąd, brak danych
brak internetu i pusta baza

Stan	Kiedy występuje
Pierwsze ładowanie	Pierwsze uruchomienie – baza jest pusta, trwa pobieranie z API.
Dane z cache	Normalny stan – ekran pokazuje dane z lokalnej bazy i ich wiek.
Odświeżanie	Trwa pobieranie nowych danych; stare są cały czas widoczne.
Błąd, dane z cache	Odświeżenie się nie udało (brak sieci, limit API), ale w bazie są dane.
Błąd, brak danych	Odświeżenie się nie udało, a baza jest pusta – przycisk „Spróbuj ponownie".

3.2 Tabela i 3.3 Mecze

Tabela pokazuje pełną klasyfikację 20 drużyn: pozycję, herb, nazwę, liczbę meczów, bilans bramek i punkty. Przełącznik u góry zmienia widok na tabelę ogólną, domową lub wyjazdową. Kolor numeru pozycji oznacza strefę: Liga Mistrzów (miejsca 1–4), Liga Europy (5) i spadek (18–20).

Mecze to terminarz i wyniki wybranej kolejki. Strzałki w nagłówku zmieniają kolejkę, a mecze są pogrupowane według dni. Ten sam komponent wiersza meczu jest używany na ekranie Start i w szczegółach wspomnienia.

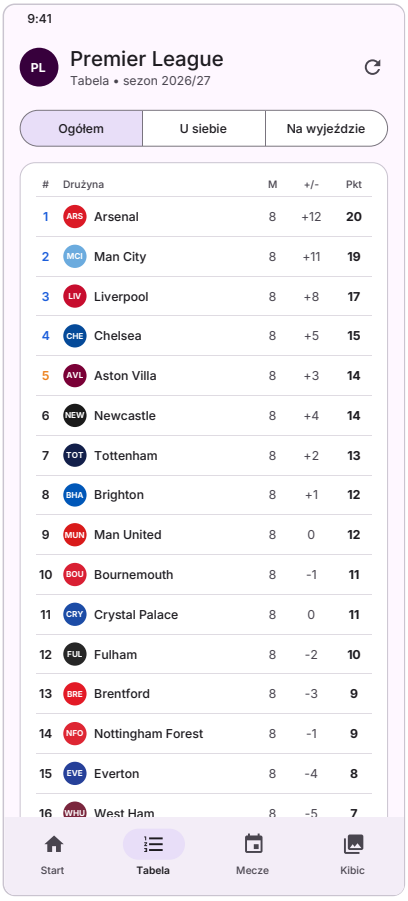
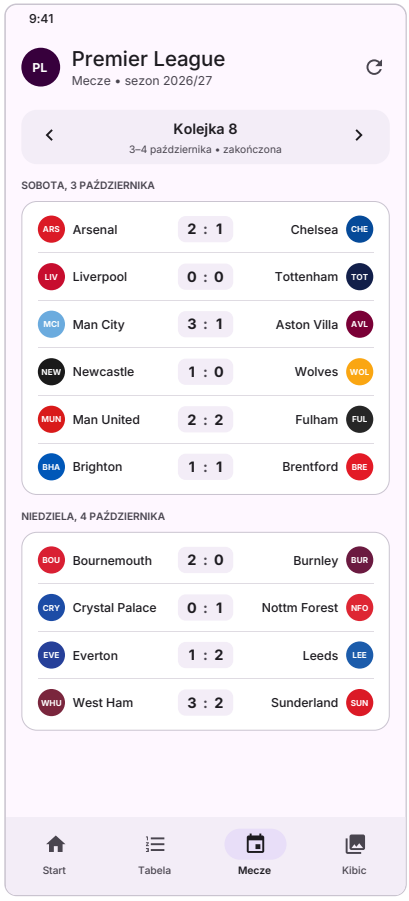


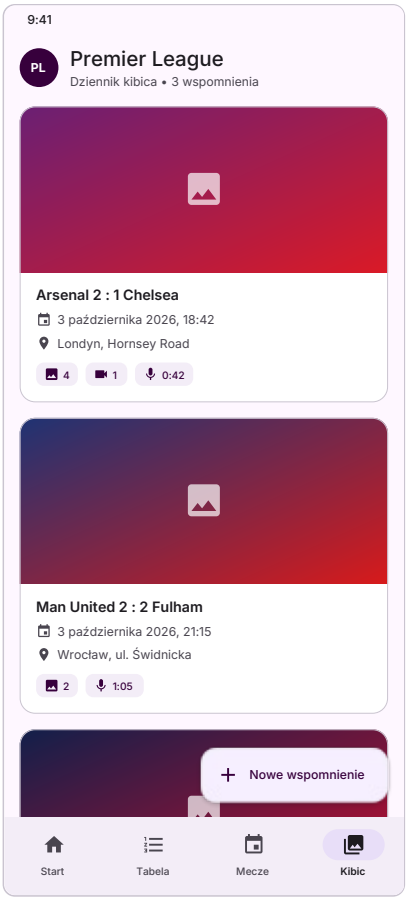
Tabela
przełącznik i strefy tabeli



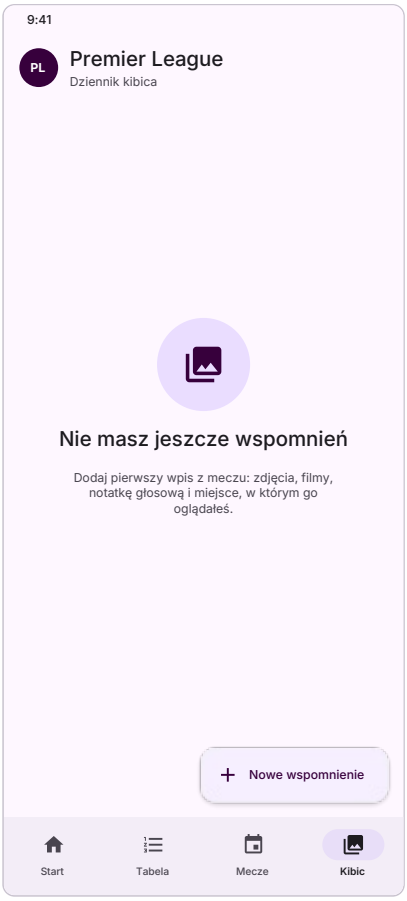
Mecze
wybór kolejki, mecze po dniach

3.4 Kibic – dziennik wspomnień

Zakładka **Kibic** to lista wspomnień użytkownika. Każda karta pokazuje miniaturę pierwszego zdjęcia, mecz, datę dodania, miejsce oraz liczniki zdjęć, filmów i długość notatki głosowej. Wpis bez lokalizacji czy notatki po prostu nie pokazuje tych elementów. Przycisk „Nowe wspomnienie" otwiera formularz.



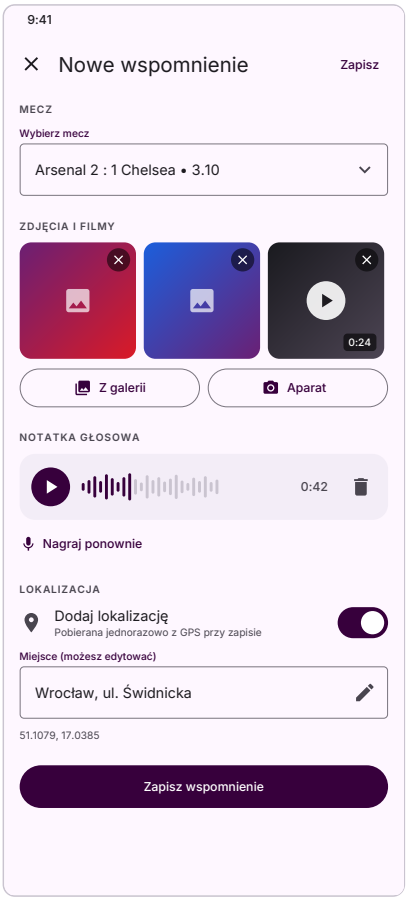
Lista wspomnień
karty wpisów + przycisk dodawania



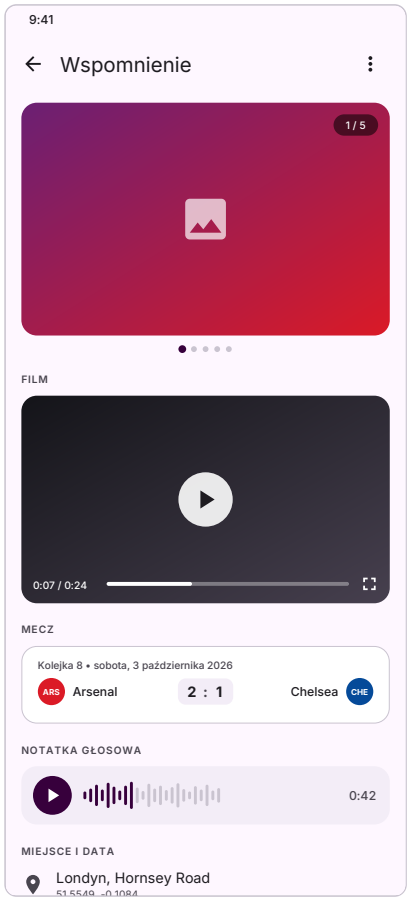
Brak wspomnień
pusty stan przy pierwszym użyciu

Formularz **nowego wspomnienia** prowadzi przez wszystkie elementy wpisu: wybór meczu, zdjęcia i filmy (z galerii lub aparatu), notatkę głosową oraz lokalizację. Lokalizacja jest opcjonalna – po włączeniu przełącznika aplikacja jednorazowo pobiera pozycję z GPS i zamienia ją na adres, który można poprawić ręcznie. Współrzędne i adres są zapisywane na stałe jako pamiątka.

Ekran **szczegółów** pokazuje wspomnienie w pełni: przewijaną galerię zdjęć, odtwarzacz filmu, kartę meczu, odtwarzacz notatki głosowej oraz miejsce i datę dodania. Z menu w prawym górnym rogu można wpis edytować lub usunąć.



Nowe wspomnienie
mecz, multimedia, notatka, GPS



Szczegóły wspomnienia
galeria, film, mecz, notatka, miejsce

Funkcja urządzenia	Uprawnienie i obsługa odmowy
Galeria (Photo Picker)	Nie wymaga uprawnień.
Aparat	Systemowa aplikacja aparatu – w praktyce bez dodatkowych uprawnień.
Mikrofon	`RECORD_AUDIO`, pytanie przy pierwszym nagraniu; po odmowie nagrywanie jest niedostępne, reszta formularza działa.
Lokalizacja	`ACCESS_COARSE_LOCATION`, pytanie po włączeniu przełącznika; po odmowie przełącznik się wyłącza i miejsce można wpisać ręcznie.

4. Plan implementacji – vertical slices

Aplikacja będzie budowana **pionowymi wycinkami (vertical slices)**. Zamiast budować najpierw całą warstwę sieci, potem całą bazę, a na końcu wszystkie ekrany, każdy etap dostarcza **jedną kompletną funkcję** – od modelu danych, przez ViewModel, po gotowy ekran. Po każdym wycinku aplikacja działa i ma o jedną funkcję więcej.

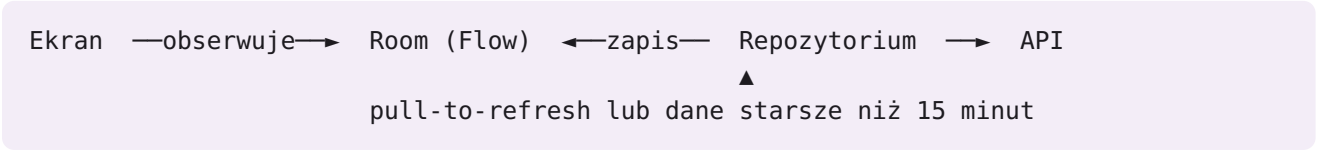
> **Dlaczego vertical slices?**
>
> - **Szybki działający efekt** – już po pierwszym wycinku jest prawdziwy, klikalny ekran, a nie same warstwy bez interfejsu.
> - **Wcześnie wychodzą błędy** – problemy z architekturą widać przy pierwszym ekranie, a nie na końcu projektu.
> - **Wzorzec do powielenia** – pierwszy wycinek (Tabela) wyznacza schemat; kolejne ekrany powtarzają go szybciej.
> - **Łatwe raportowanie postępu** – każdy etap kończy się czymś, co można uruchomić i pokazać.

Kolejność jest celowa: najpierw powstaje **cały model danych i mocki**, potem kolejne ekrany działające na mockach, następnie baza i dziennik, a **podłączenie API odbywa się na końcu**. Ekran Start jest ostatnim wycinkiem ligowym, bo łączy dane z kilku repozytoriów i korzysta z gotowych komponentów pozostałych ekranów. Plan jest również narysowany w Figmie jako stepper (strona „Plan implementacji”).

#	Faza	Etap	Zakres	Efekt
0	Przygotowanie	Projekt i dokumentacja	Makiety w Figmie, diagramy architektury, ten dokument.	Materiały do akceptacji
1	Przygotowanie	Fundament aplikacji	Projekt w Android Studio, git, zależności, motyw z Figmy, `Scaffold` + nawigacja z 4 pustymi zakładkami, Hilt.	Szkielet z działającą nawigacją
2	Przygotowanie	Cały model i mocki	Model domenowy (Team, Match, StandingRow, StandingType, Competition, Memory), interfejsy repozytoriów, fake repozytoria z danymi z makiet i symulacją opóźnień oraz błędów.	Aplikacja zna wszystkie dane, zero zapytań HTTP
3	Wycinki na mockach	Wycinek 1: Tabela	ViewModel + UiState, wiersz tabeli, przełącznik, strefy, wszystkie stany, test ViewModelu.	Pierwsza zakładka – wzorzec
4	Wycinki na mockach	Wycinek 2: Mecze	Wybór kolejki, grupowanie po dniach, wiersz meczu, stany, test.	Druga zakładka
5	Wycinki na mockach	Wycinek 3: Start	ViewModel łączący trzy repozytoria, sekcje podsumowania, linki do zakładek.	Kompletna część ligowa na mockach
6	Dziennik kibica	Baza Room	Konfiguracja bazy z eksportem schematu, encje wspomnień (relacja 1:N z multimediami), DAO.	Trwałe miejsce na dane użytkownika
7a	Dziennik kibica	Wycinek 4a: lista i zdjęcia	Lista i pusty stan, formularz z wyborem meczu, zdjęcia z galerii i aparatu, szczegóły, edycja i usuwanie.	Działający dziennik ze zdjęciami
7b	Dziennik kibica	Wycinek 4b: wideo, audio, GPS	Filmy i odtwarzacz Media3, nagrywanie i odtwarzanie notatki, lokalizacja z GPS + Geocoder, uprawnienia.	Spełnione wymagania multimedialne
8	Prawdziwe dane	Podłączenie API	DTO, Retrofit, interceptor z kluczem, mappery, prawdziwe repozytoria implementujące te same interfejsy, warianty mock/prod.	Prawdziwe dane bez zmian w UI
9	Prawdziwe dane	Cache offline-first	Tabele cache dodane migracją (`@AutoMigration`), reguła 15 minut, pull-to-refresh, obsługa braku sieci i limitu API.	Działa offline, dziennik nienaruszony
10	Oddanie	Testy i oddanie	Testy repozytoriów i ViewModeli, test migracji bazy, testy ręczne (tryb samolotowy, odmowa uprawnień), README.	Projekt gotowy do oddania

5. Dane offline – podejście cache-first

Aplikacja działa w modelu **offline-first (cache-first)**. Oznacza to, że **ekran nigdy nie czyta danych bezpośrednio z API**. Zawsze czyta je z lokalnej bazy danych Room, a API służy wyłącznie do jej aktualizowania. Lokalna baza jest więc jedynym „źródłem prawdy" dla interfejsu.



Jak to działa krok po kroku

1. **Wejście na ekran** – dane z bazy pojawiają się natychmiast, bez czekania na sieć.
2. **Sprawdzenie świeżości** – repozytorium sprawdza czas ostatniej aktualizacji (zapisany w bazie). Jeśli dane są starsze niż 15 minut, w tle wysyła zapytanie do API.
3. **Odpowiedź z API** – nowe dane są zapisywane do bazy. Room sam powiadamia ekran o zmianie (strumień `Flow`), więc widok odświeża się automatycznie.
4. **Pull-to-refresh** – przeciągnięcie listy w dół wymusza pobranie danych niezależnie od ich wieku.
5. **Błąd sieci lub limitu API** – dane z bazy zostają na ekranie, a użytkownik widzi komunikat „Nie udało się odświeżyć" z opcją ponowienia. Pusty ekran błędu pojawia się tylko wtedy, gdy baza jest pusta.

Co to daje

- **Działanie bez internetu** – po pierwszym pobraniu aplikacja pokazuje tabelę i mecze także w trybie samolotowym, z informacją o wieku danych.
- **Oszczędzanie limitu API** – darmowy plan pozwala na 10 zapytań na minutę; reguła 15 minut sprawia, że zwykłe przełączanie zakładek w ogóle nie zużywa limitu.
- **Szybkość** – ekran nie czeka na sieć, więc dane są widoczne od razu.
- **Prosty UI** – ekran zawsze ma jedno źródło danych, niezależnie od stanu sieci.

Trwałość danych

Baza Room to plik SQLite w prywatnym katalogu aplikacji. Dane przetrwają zamknięcie aplikacji, restart telefonu oraz jej aktualizację (także kolejne uruchomienia z Android Studio). Znikają dopiero po odinstalowaniu aplikacji lub wyczyszczeniu jej danych w ustawieniach.

> **Jedna baza, dwa rodzaje danych.** W tej samej bazie są dane ligowe (cache, który zawsze można pobrać ponownie) i wspomnienia z dziennika (dane użytkownika, których nie wolno utracić). Dlatego zmiany schematu bazy nie mogą jej po prostu kasować. Tabele cache zostaną dodane w etapie 9 przez migrację `@AutoMigration`, która zachowuje istniejące wspomnienia.

6. Mocki jako zabezpieczenie projektu

Przez większość projektu aplikacja działa na **danych testowych (mockach)** zamiast na prawdziwym API. Każde repozytorium jest zdefiniowane jako **interfejs**, a aplikacja ma dwie jego implementacje:

- **Fake...Repository** – zwraca dane przygotowane na podstawie makiet, z konfigurowalnym opóźnieniem i symulacją błędów,
- **Default...Repository** – prawdziwa implementacja oparta o API football-data.org i bazę Room.

Ekrany i ViewModele znają wyłącznie interfejs, więc nie wiedzą, która implementacja została użyta. Wyboru dokonuje Gradle przez dwa warianty budowania (**product flavors**): `mock` i `prod`. Hilt wstrzykuje do aplikacji tę implementację, która należy do wybranego wariantu.

> **Zabezpieczenie terminu.** Podłączenie API jest jednym z ostatnich etapów (8). Jeśli zabraknie czasu na jego dokończenie, aplikacja i tak będzie kompletna i w pełni funkcjonalna na danych testowych – wszystkie ekrany, stany i Dziennik kibica działają niezależnie od API. Prezentacja projektu nie zależy więc od ostatniego etapu.

Dodatkowe korzyści z mocków

- **Niezależność od internetu i limitu API** – podczas tworzenia UI nie zużywa się limitu 10 zapytań na minutę.
- **Testowanie wszystkich stanów** – fake repozytorium może celowo opóźnić odpowiedź lub zwrócić błąd, więc stany ładowania i błędu można sprawdzić bez wyłączania internetu.
- **Stabilna prezentacja** – wariant `mock` pokazuje zawsze te same, przewidywalne dane, niezależnie od sieci na sali i od tego, czy trwa sezon.
- **Testy jednostkowe** – ViewModele testuje się na fake'ach, bez emulatora i bez sieci.
- **Podmiana bez zmian w UI** – w etapie 8 zmienia się tylko implementacja repozytoriów; ekrany i ViewModele zostają bez zmian.

Oba warianty mają różny `applicationId` (z przyrostkiem `.mock`), więc mogą być zainstalowane na telefonie jednocześnie.

7. Technologie

Obszar	Technologia	Zastosowanie
Język i UI	Kotlin, Jetpack Compose, Material 3	Ekrany i komponenty interfejsu
Nawigacja	Navigation Compose	Przejścia między zakładkami i ekranami
Architektura	ViewModel, StateFlow, Coroutines	Stan ekranów, operacje w tle
Wstrzykiwanie zależności	Hilt	Spinanie warstw, wybór mock / prod
Sieć	Retrofit, OkHttp, kotlinx.serialization	Zapytania do API i parsowanie JSON
Baza danych	Room (SQLite)	Cache danych ligowych i wspomnienia
Obrazy	Coil	Herby klubów, zdjęcia ze wspomnień
Multimedia	Media3 (ExoPlayer), MediaRecorder, Photo Picker	Film, notatka głosowa, wybór plików
Lokalizacja	Fused Location Provider, Geocoder	Pozycja i adres przy wspomnieniu
Testy	JUnit, MockK, Turbine	Testy repozytoriów i ViewModeli

8. Architektura MVVM – diagramy

Aplikacja jest zbudowana w architekturze **MVVM (Model–View–ViewModel)**, zalecanej przez Google dla aplikacji na Androida. Kod jest podzielony na trzy warstwy o jasno określonych odpowiedzialnościach:

Warstwa	Odpowiedzialność	W projekcie
View	Wyświetla stan i przekazuje akcje użytkownika. Nie zawiera logiki.	Ekrany Compose: Start, Tabela, Mecze, ekrany dziennika, wspólny szkielet aplikacji.
ViewModel	Trzyma stan ekranu (`UiState`), reaguje na akcje, woła Model. Przeżywa obrót ekranu.	StartViewModel, StandingsViewModel, MatchesViewModel, ViewModele dziennika.
Model	Dane i logika: modele domenowe, repozytoria i źródła danych.	*Domena:* modele i interfejsy repozytoriów. *Dane:* implementacje fake/prod, Retrofit, Room, API urządzenia.

Dwie najważniejsze zasady

- **Zależności idą w dół.** View zna ViewModel, ViewModel zna tylko *interfejs* repozytorium. Implementacje z warstwy danych realizują te interfejsy, więc ViewModel nie wie, czy dane pochodzą z mocka, API czy bazy.
- **Stan płynie w górę, zdarzenia w dół.** Kliknięcia i gesty idą od ekranu do ViewModelu jako wywołania funkcji. Dane wracają jako strumienie: Room i repozytoria zwracają `Flow`, a ViewModel wystawia ekranowi `StateFlow<UiState>`. Ekran obserwuje stan i przerysowuje się sam, gdy ten się zmieni.

Diagramy

Na kolejnych stronach znajdują się dwa diagramy, narysowane również w Figmie (strona „Diagramy”):

- **Diagram A – przegląd całej aplikacji.** Wszystkie ekrany, ViewModele, repozytoria i źródła danych rozmieszczone w warstwach View, ViewModel i Model, razem z systemami zewnętrznymi i rolą Hilt. Pary strzałek między warstwami pokazują akcje w dół i stan w górę.
- **Diagram B – zbliżenie na wycinek „Tabela”.** Konkretne klasy na całej drodze od ekranu do API oraz ponumerowany przepływ danych przy odświeżaniu tabeli. Diagram pełni też rolę diagramu sekwencji – opis kroków znajduje się obok niego.

> **Jak czytać strzałki.** Linia ciągła oznacza wywołanie lub zależność („kto używa kogo”). Fioletowa linia przerywana oznacza przepływ danych w górę (`Flow`, `StateFlow`). Szara linia przerywana oznacza implementację interfejsu. Ramka przerywana wyróżnia implementację testową (wariant mock).

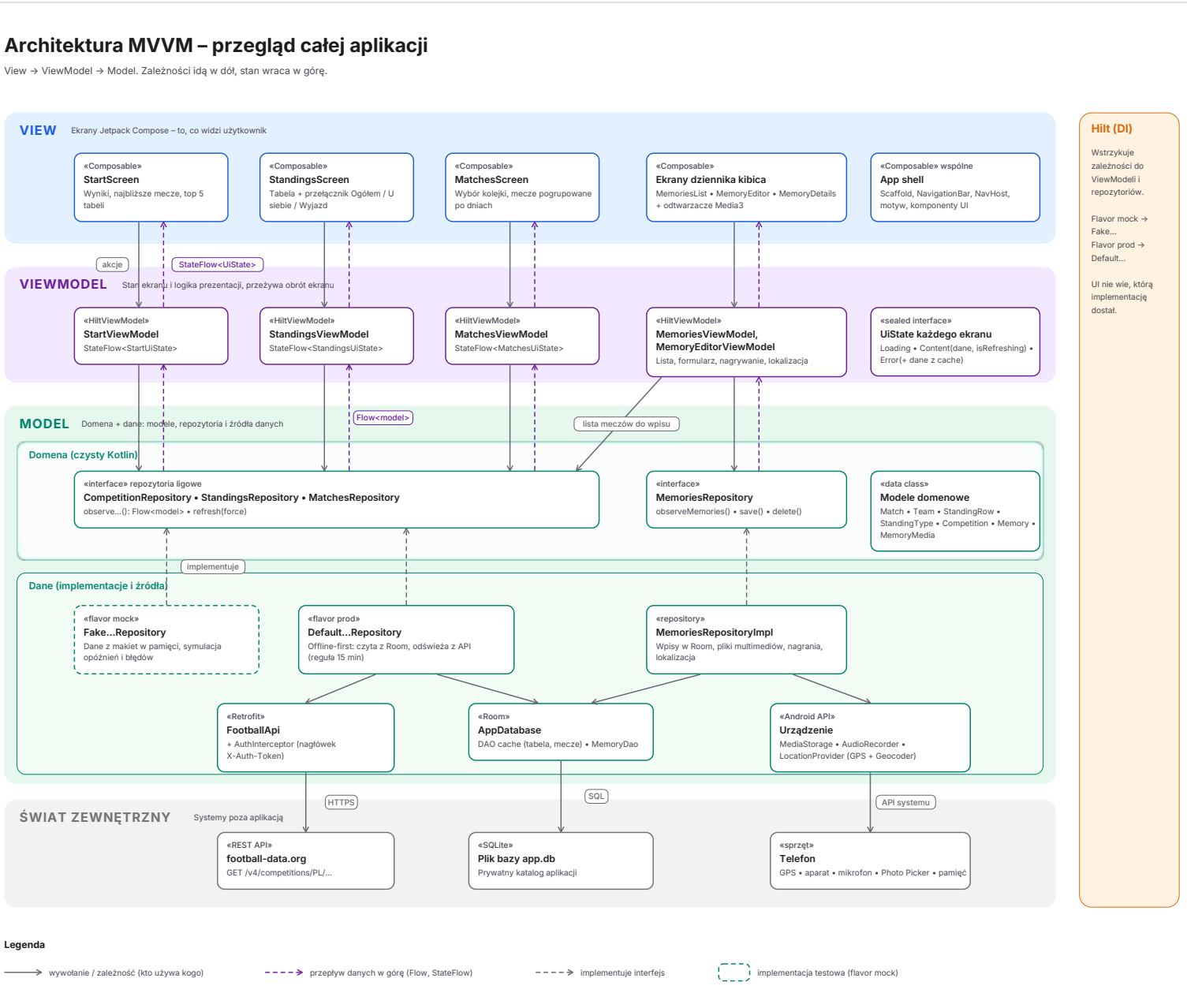


Diagram A – przegląd

VIEW – ekrany Compose każdej zakładki oraz wspólny szkielet aplikacji (Scaffold, nawigacja, motyw).

VIEWMODEL – osobny ViewModel dla każdego ekranu. Każdy wystawia `StateFlow<UiState>` ze stanami Loading / Content / Error.

MODEL – domena – interfejsy repozytoriów i modele domenowe. To czysty Kotlin, bez zależności od Androida.

MODEL – dane – dwie implementacje repozytoriów ligowych: *Fake* (wariant mock) i *Default* (wariant prod, offline-first), repozytorium dziennika oraz źródła: Retrofit, Room i API urządzenia.

Świat zewnętrzny – football-data.org, plik bazy oraz sprzęt telefonu.

Hilt spina wszystkie warstwy i w zależności od wariantu budowania wstrzykuje implementację mock lub prod.

Dziennik kibica korzysta z repozytorium meczów (strzałka „lista meczów do wpisu"), ale nie wymaga API – jego dane żyją w Room i w plikach urządzenia.

Wycinek „Tabela” – przepływ danych przy odświeżaniu

Jedna funkcja od ekranu po API. Ten sam wzorzec powtarzają Mecze, Start i Dziennik kibica.

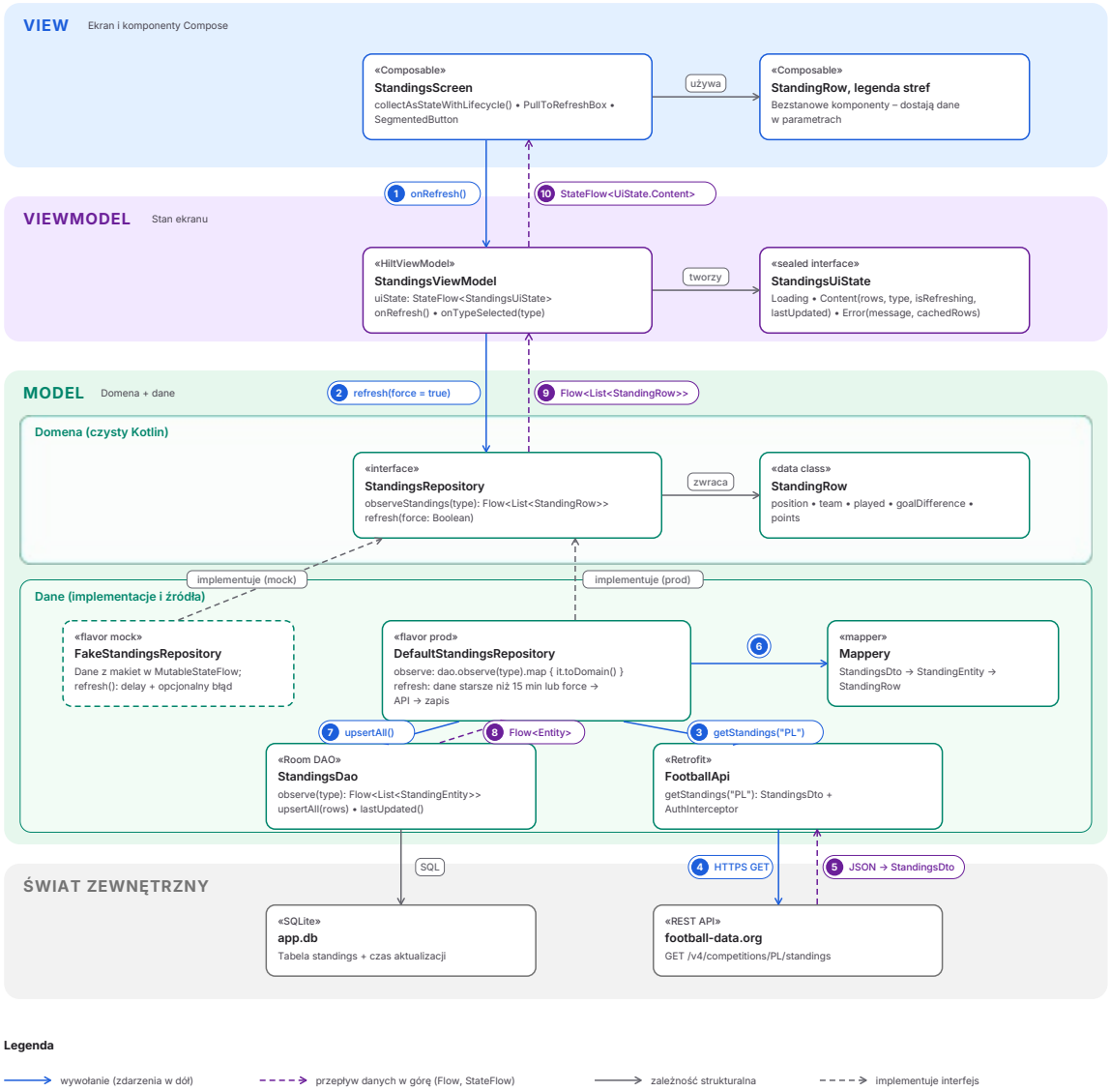


Diagram B – przepływ pull-to-refresh na Tabeli

1. Użytkownik przeciąga listę w dół – ekran wywołuje `viewModel.onRefresh()`.
2. ViewModel ustawia `isRefreshing = true` i woła `repository.refresh(force = true)`.
3. `DefaultStandingsRepository` pyta `FootballApi` o tabelę.
4. Retrofit wysyła żądanie HTTPS GET z nagłówkiem `X-Auth-Token`.
5. API zwraca JSON, który kotlinx.serialization zamienia na `StandingsDto`.
6. Mapper zamienia DTO na encje bazy.
7. Repozytorium zapisuje encje w Room (`upsertAll`) razem z czasem aktualizacji.
8. Room sam emituje nową listę przez `Flow`, bo dane w tabeli się zmieniły.
9. Repozytorium mapuje encje na model domenowy `StandingRow`.
10. ViewModel składa `StandingsUiState.Content` i wystawia go w `StateFlow`.
11. Compose rekomponuje ekran – widać nową tabelę.

Błąd sieci (kroki 4–5) – repozytorium zgłasza wyjątek, a ViewModel pokazuje dane z cache i snackbar; gdy cache jest pusty – stan Error.

Wariant mock – kroki 3–8 zastępuje `FakeStandingsRepository`; kroki 1, 2 i 9–11 wyglądają identycznie.

Pozostałe ekrany – Mecze, Start i Dziennik kibica działają według tego samego wzorca.